GenAI Project

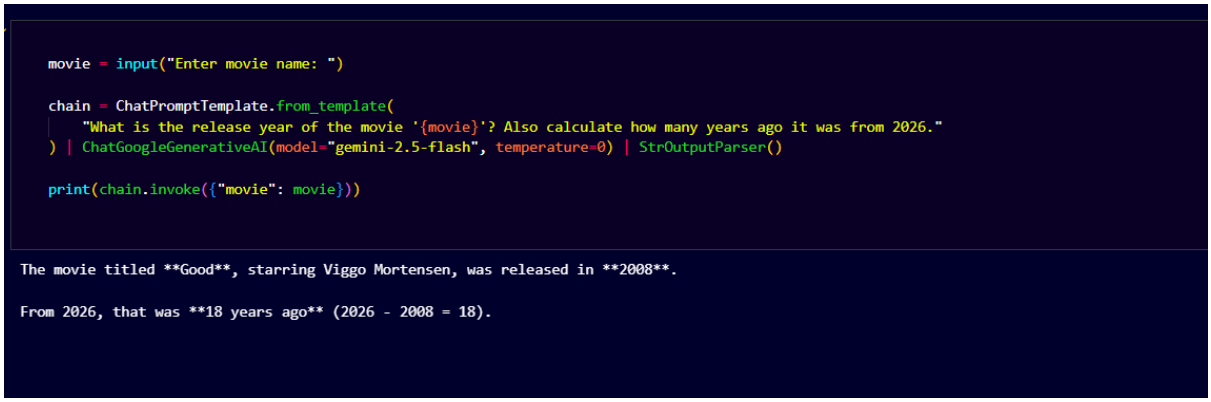
Name: Deesha C
SRN: PES2UG23CS165
SEC: C

1.Lang Chain Foundation

Assignment

Create a chain that:

1. *Takes a movie name.*
2. *Asks for its release year.*
3. *Calculates how many years ago that was (You can try just asking the LLM to do the math).*



```
movie = input("Enter movie name: ")

chain = ChatPromptTemplate.from_template(
    "What is the release year of the movie '{movie}'? Also calculate how many years ago it was from 2026."
) | ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0) | StrOutputParser()

print(chain.invoke({"movie": movie}))
```

```
The movie titled **Good**, starring Viggo Mortensen, was released in **2008**.

From 2026, that was **18 years ago** (2026 - 2008 = 18).
```

Abstract

This project demonstrates the use of LangChain Expression Language (LCEL) to build a simple composable AI chain. The system accepts a movie name from the user, queries the LLM to determine the movie's release year, and then computes how many years ago the movie was released relative to the year 2026.

The goal of the assignment is to understand how to design a minimal yet functional AI pipeline using prompt templates, Google Generative AI, and output parsers in a single LCEL expression. The project highlights how AI logic can be modular, reusable, and composable using LangChain.

Understanding:

From this assignment, I understood that:

- LCEL allows chaining multiple AI operations in a clean pipeline.
- A prompt template can dynamically insert user input.
- The LLM can be used not only for text generation but also for reasoning tasks like year calculation.
- Output parsers help convert the LLM response into usable text.
- The entire workflow can be composed in **one line**, showing the power of LangChain composability.

What I Built:
Accepts a movie name from the user.
Uses ChatPromptTemplate to frame the question.
Sends the prompt to the Gemini model (gemini-2.5-flash).
The model finds the movie's release year and calculates how many years ago it was from 2026.
The result is parsed using StrOutputParser and printed.

2. Prompt Engineering

Assignment

*Write a structured prompt to generate a **Python Function**.*
*- **Context:** You are a Senior Python Dev.*
*- **Objective:** Write a function to reverse a string.*
*- **Constraint:** It must use recursion (no slicing `[::-1]`).*
*- **Style:** Include detailed docstrings.*

```
structured_prompt = """
# Context
You are a Senior Python Developer who writes clean, production-ready code
and follows best practices.

# Objective
Write a Python function that reverses a string.

# Constraints
1. The function MUST use recursion.
2. Do NOT use slicing (e.g., [::-1]).
3. Do NOT use any built-in reverse methods.
4. Include proper type hints.
5. Handle invalid input with appropriate exceptions.

# Style
- Include detailed and professional docstrings.
- Follow clean coding principles.
- Keep the implementation readable and maintainable.

# Output Format
Return only valid Python code.
"""

print("--- STRUCTURED PROMPT ---")
print(llm.invoke(structured_prompt).content)
```

[20]

```
--- STRUCTURED PROMPT ---
```python
import sys

def reverse_string(s: str) -> str:
    """
    Reverses a given string using a recursive approach.

    This function takes an input string and returns a new string with its
    characters in reverse order. It strictly adheres to a recursive implementation
    and avoids the use of string slicing for reversal (e.g., [::-1]) or
    any built-in reverse methods (e.g., reversed()).

    Args:
        s: The input string to be reversed.

    Returns:
        A new string with the characters of the input string in reverse order.

    Raises:
        TypeError: If the input `s` is not a string.

    Examples:
        >>> reverse_string("hello")
        'olleh'
...
        #    By placing the first character at the end, we effectively reverse its position.
        return reverse_string(s[1:]) + s[0]

```
```

Abstract

This assignment focuses on prompt engineering to generate high-quality Python code using a structured prompt. The goal is to guide a Large Language Model (LLM) to produce a clean, production-ready recursive function for reversing a string.

By clearly specifying context, objective, constraints, and style requirements, the structured prompt ensures that the generated code follows best practices such as type hints, exception handling, and detailed documentation. This exercise demonstrates how well-designed prompts can significantly improve the reliability and quality of AI-generated code.

Understanding

From this task, I understood that:

- Structured prompting helps control LLM output quality.
- Providing **role context** (Senior Python Developer) improves professionalism of generated code.
- Clearly defined constraints prevent the model from using shortcuts like slicing.
- Style instructions ensure maintainable and production-ready output.
- Prompt engineering is critical for predictable code generation.

What I Built

The prompt includes:

- Clear developer role context
- Specific functional objective
- Strict technical constraints
- Professional coding style requirements
- Output formatting instructions

When executed, the LLM generates a well-documented recursive function that:

- Uses pure recursion
- Avoids slicing and built-ins
- Includes type hints
- Handles invalid inputs
- Follows clean coding practices